Programmation Frontend

Lab 6 — Consommer une API Django

CORS — authentification — CRUD complet

ESP/UCAD — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	2h (une séance)
Prérequis	Avoir terminé le Lab 5 (Next.js App Router). Python 3.10+ installé.
Fichiers de départ	Votre projet `annuaire-next/` du Lab 5 + le script `setup_django.sh` fourni
Livrables	Annuaire Next.js connecté au backend Django avec login et CRUD
Validation	Les données sont persistées : ajouter un étudiant, recharger, il est toujours là.

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. Expliquer ce qu'est **CORS** et pourquoi le navigateur bloque certaines requêtes
2. Utiliser des **variables d'environnement** pour configurer l'URL de l'API
3. Centraliser les appels API dans un module `lib/api.js`
4. Implémenter l'**authentification par token** (login + envoi dans les headers)
5. Réaliser un **CRUD complet** (Create, Read, Delete) avec des données persistées

Table des matières

1 Partie 1 — Installer et explorer le backend **3**
1.1 Lancer le backend . . . 3
1.2 Les endpoints de l'API . . . 3
1.3 Tester avec `curl` . . . 3

2 Partie 2 — Comprendre CORS **4**

3 Partie 3 — Préparer le frontend **4**
3.1 Étape 1 — Variable d'environnement . . . 4
3.2 Étape 2 — Module `lib/api.js` . . . 5

3.3 Étape 3 — Gestion du token 6

4 Partie 4 — Page de login **7**
4.1 Créer le dossier et la page 7
4.2 Tester le login 9

5 Partie 5 — Modifier les pages existantes **9**
5.1 Modifier `UserCard.jsx` 9
5.2 Modifier la page de liste 10
5.3 Modifier la page de détail 12
5.4 Modifier le formulaire d'ajout 14
5.5 Modifier la Navbar 15

6 Partie 6 — Tester la persistance **15**

7 Aide-mémoire **16**

8 Résumé et conclusion **17**

1 Partie 1 — Installer et explorer le backend

Objectif — 10 minutes

Lancer le backend Django fourni et tester l'API avec `curl`.

1.1 Lancer le backend

L'enseignant vous fournit un script `setup_django.sh`. Ce script crée un projet Django complet avec une base de données pré-remplie de 10 étudiants sénégalais.

```
chmod +x setup_django.sh
./setup_django.sh
```

Puis lancez le serveur Django :

```
cd annuaire-api
source venv/bin/activate
python manage.py runserver
```

Le serveur tourne sur `http://localhost:8000`. **Gardez ce terminal ouvert** pendant tout le lab.

1.2 Les endpoints de l'API

Méthode	URL	Description	Auth
POST	`/api/auth/login/`	Connexion → token	Non
GET	`/api/students/`	Liste des étudiants	Token
POST	`/api/students/`	Créer un étudiant	Token
GET	`/api/students/3/`	Détail étudiant 3	Token
DELETE	`/api/students/3/`	Supprimer étudiant 3	Token

1.3 Tester avec `curl`

Ouvrez un **deuxième terminal**. Commencez par le login :

```
curl -X POST http://localhost:8000/api/auth/login/ \
  -H "Content-Type: application/json" \
  -d '{"username":"prof","password":"le mot de passe défini dans
     setup_django.sh"}'
```

Résultat :

```
{"token":"abc123def456...","username":"prof"}
```

Copiez le token. Utilisez-le pour accéder aux données :

```
curl http://localhost:8000/api/students/ \
  -H "Authorization: Token abc123def456..."
```

Vous devez voir la liste des 10 étudiants au format JSON : Fatou Ndiaye, Moussa Diop, Aminata Sow...

Sans le token — erreur 401

Si vous oubliez le header `Authorization`, le backend répond :

```
{"detail":"Authentication credentials were not provided."}
```

L'API est protégée. Chaque requête (sauf le login) doit envoyer le token.

Le modèle Student

Chaque étudiant a ces champs : `id`, `prenom`, `nom`, `email`, `filiere` (IABD, SIR), `niveau` (L1 à M2), `date_naissance`, `telephone`.
C'est **différent** du modèle JSONPlaceholder (`name`, `email`, `company.name`). On va adapter le frontend.

2 Partie 2 — Comprendre CORS

Objectif — 5 minutes

Comprendre pourquoi le navigateur bloque les requêtes entre origines différentes.

Le frontend tourne sur `localhost:3000`. Le backend sur `localhost:8000`. Ce sont deux **origines différentes** (ports différents). Par défaut, le navigateur **interdit** à un site d'envoyer des requêtes vers une autre origine. C'est la politique CORS (*Cross-Origin Resource Sharing*).

Analogie : le gardien de l'immeuble

Quand quelqu'un sonne, le gardien vérifie la liste des visiteurs autorisés. Si le visiteur n'est pas sur la liste, il est refusé. CORS fonctionne pareil : le backend maintient une liste d'origines autorisées.

Le backend fourni a déjà `django-cors-headers` configuré : toutes les origines sont autorisées (mode développement). Vous n'avez rien à faire côté backend. En production, on limiterait la liste.

3 Partie 3 — Préparer le frontend

Objectif — 15 minutes

Créer la variable d'environnement, le module API et la gestion du token.

Ouvrez votre projet `annuaire-next/` du Lab 5 dans VS Code. Lancez le serveur dans un **troisième terminal** :

```
cd annuaire-next
npm run dev
```

Vérifiez que l'annuaire Lab 5 fonctionne sur `http://localhost:3000`.

3.1 Étape 1 — Variable d'environnement

À la **racine** du projet (pas dans `src/`), créez un fichier `.env.local` :

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api
```

Variables d'environnement Next.js

Préfixe	Accès
`NEXT_PUBLIC_`	Accessible côté client et serveur
Sans préfixe	Serveur uniquement

L'URL de l'API est utilisée dans des composants `"use client"`. Elle a donc besoin du préfixe `NEXT_PUBLIC_`.
Au lieu d'écrire l'URL en dur dans chaque `fetch`, on écrira `process.env.NEXT_PUBLIC_API_URL`. Si l'URL change (en production par exemple), on modifie un seul fichier.

3.2 Étape 2 — Module `lib/api.js`

Créez le dossier `src/lib/` et le fichier `src/lib/api.js` :

```
mkdir -p src/lib
```

```
// src/lib/api.js
const API = process.env.NEXT_PUBLIC_API_URL;

export const login = async (username, password) => {
  const res = await fetch(`${API}/auth/login/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password }),
  });
  if (!res.ok) {
    const data = await res.json().catch(() => ({}));
    throw new Error(
      data.error || "Identifiants invalides"
    );
  }
  return res.json();
};

export const getStudents = async (token) => {
  const res = await fetch(`${API}/students/`, {
    headers: { Authorization: `Token ${token}` },
  });
  if (!res.ok)
    throw new Error("Erreur de chargement");
  return res.json();
};

export const getStudent = async (token, id) => {
  const res = await fetch(`${API}/students/${id}/`, {
    headers: { Authorization: `Token ${token}` },
  });
  if (!res.ok)
    throw new Error("Étudiant introuvable");
```

```
  return res.json();
};

export const createStudent = async (token, data) => {
  const res = await fetch(`${API}/students/`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Token ${token}`,
    },
    body: JSON.stringify(data),
  });
  if (!res.ok)
    throw new Error("Erreur de création");
  return res.json();
};

export const deleteStudent = async (token, id) => {
  const res = await fetch(`${API}/students/${id}/`, {
    method: "DELETE",
    headers: { Authorization: `Token ${token}` },
  });
  if (!res.ok)
    throw new Error("Erreur de suppression");
};
```

Pourquoi centraliser les appels API ?

Au Lab 5, les URL JSONPlaceholder étaient écrites directement dans les composants. Si l'URL change, il faut modifier chaque fichier. Avec `lib/api.js`, on change une seule ligne (`.env.local`). Chaque fonction correspond à une opération du CRUD.

3.3 Étape 3 — Gestion du token

Créez `src/lib/auth.js` :

```
// src/lib/auth.js
"use client";

export const getToken = () => {
  if (typeof window === "undefined") return null;
  return sessionStorage.getItem("token");
};

export const setToken = (token) => {
  sessionStorage.setItem("token", token);
};

export const removeToken = () => {
  sessionStorage.removeItem("token");
};

export const isLoggedIn = () => {
  return !!getToken();
};
```

Le token est stocké dans `sessionStorage`. Il disparaît quand l'onglet est fermé (plus sûr que `localStorage`).

4 Partie 4 — Page de login

Objectif — 15 minutes

Créer une page de connexion et protéger les pages.

4.1 Créer le dossier et la page

```
mkdir -p src/app/login
```

Créez `src/app/login/page.js` :

```
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import { login } from "@/lib/api";
import { setToken } from "@/lib/auth";
import styles from "./page.module.css";

export default function LoginPage() {
  const router = useRouter();
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [erreur, setErreur] = useState(null);
  const [enCours, setEnCours] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setErreur(null);
    setEnCours(true);

    try {
      const data = await login(username, password);
      setToken(data.token);
      router.push("/users");
    } catch (error) {
      setErreur(error.message);
    }
    setEnCours(false);
  };

  return (
    <div>
      <h1 className={styles.titre}>Connexion</h1>
      <div className={styles.carte}>
        <form onSubmit={handleSubmit}>
          <div className={styles.groupe}>
            <label className={styles.label}
                   htmlFor="username">
              Nom d'utilisateur
            </label>
            <input className={styles.input}
```

```
              type="text" id="username"
              placeholder="prof"
              value={username}
              onChange={(e) =>
                setUsername(e.target.value)
              }
            />
          </div>
          <div className={styles.groupe}>
            <label className={styles.label}
                   htmlFor="password">
              Mot de passe
            </label>
            <input className={styles.input}
              type="password" id="password"
              placeholder="mot de passe"
              value={password}
              onChange={(e) =>
                setPassword(e.target.value)
              }
            />
          </div>
          {erreur && (
            <p className={styles.erreur}>{erreur}</p>
          )}
          <button type="submit"
                  className={styles.bouton}
                  disabled={enCours}>
            {enCours ? "Connexion..." :
                       "Se connecter"}
          </button>
        </form>
        <p className={styles.hint}>
          Identifiants de test : prof / le mot de passe affiché par
             setup_django.sh
        </p>
      </div>
    </div>
  );
}
```

Créez `src/app/login/page.module.css` :

```
.titre { font-size: 1.5rem; color: #2d3436;
         margin-bottom: 1.5rem; }
.carte { background: #fff;
         border: 1px solid #dfe6e9;
         border-radius: 10px; padding: 1.5rem;
         max-width: 400px; }
.groupe { margin-bottom: 1.25rem; }
.label { display: block; font-size: 0.9rem;
         font-weight: 600; color: #2d3436;
         margin-bottom: 0.4rem; }
.input { width: 100%; padding: 0.65rem 0.9rem;
         font-size: 0.95rem;
         border: 1px solid #dfe6e9;
         border-radius: 8px; }
.input:focus { outline: none;
```

```
                border-color: #0984e3; }
.erreur { color: #d63031; font-weight: 600;
          padding: 0.75rem; background: #ffeaa7;
          border-radius: 6px;
          margin-bottom: 1rem; }
.bouton { width: 100%; padding: 0.75rem;
          background: #0984e3; color: #fff;
          border: none; border-radius: 8px;
          font-size: 1rem; font-weight: 600;
          cursor: pointer; }
.bouton:hover { background: #0770c2; }
.bouton:disabled { background: #b2bec3;
                   cursor: not-allowed; }
.hint { color: #b2bec3; font-size: 0.8rem;
        text-align: center; margin-top: 1rem; }
```

4.2 Tester le login

Allez sur `http://localhost:3000/login`. Saisissez `prof` et le mot de passe que `setup_django.sh` a affiché. Si la connexion réussit, vous êtes redirigé vers `/users`. Si vous tapez un mauvais mot de passe, le message d'erreur s'affiche en rouge.

Redémarrer le serveur Next.js

Après avoir créé `.env.local`, il faut **arrêter** et **relancer** `npm run dev`. Next.js ne recharge pas les variables d'environnement à chaud.

5 Partie 5 — Modifier les pages existantes

Objectif — 25 minutes

Adapter la liste, le détail, l'ajout et la Navbar pour utiliser l'API Django.

Les modifications suivent un principe simple : remplacer les appels directs à JSONPlaceholder par les fonctions de `lib/api.js`, et adapter les champs au modèle Student (`prenom`, `nom`, `filiere`... au lieu de `name`, `company.name`).

5.1 Modifier `UserCard.jsx`

Le composant reçoit maintenant un objet `student` (pas `user`). Remplacez le contenu de `src/components/UserCard.jsx` :

```
"use client";

import Link from "next/link";
import styles from "./UserCard.module.css";

const UserCard = ({ student }) => {
  return (
    <div className={styles.card}>
      <div className={styles.header}>
        <Link href={`/users/${student.id}`}
              className={styles.lien}>
          <h3 className={styles.nom}>
```

```
            {student.prenom} {student.nom}
          </h3>
        </Link>
        <span className={styles.badge}>
          {student.filiere} {student.niveau}
        </span>
      </div>
      <p className={styles.info}>
        Email : {student.email}
      </p>
      {student.telephone && (
        <p className={styles.info}>
          Tél. : {student.telephone}
        </p>
      )}
      <Link href={`/users/${student.id}`}
            className={styles.voir}>
        Voir le profil →
      </Link>
    </div>
  );
};

export default UserCard;
```

Ajoutez la classe `.badge` dans `UserCard.module.css` :

```
.badge {
  background: #dfe6e9;
  color: #2d3436;
  padding: 0.2rem 0.6rem;
  border-radius: 12px;
  font-size: 0.75rem;
  font-weight: 600;
}
```

5.2 Modifier la page de liste

Remplacez le contenu de `src/app/users/page.js` :

```
"use client";

import { useState, useEffect } from "react";
import { useRouter } from "next/navigation";
import { getStudents } from "@/lib/api";
import { getToken, isLoggedIn } from "@/lib/auth";
import UserCard from "@/components/UserCard";
import SearchBar from "@/components/SearchBar";
import styles from "./page.module.css";

export default function UserList() {
  const router = useRouter();
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);
  const [recherche, setRecherche] = useState("");
  const [triAscendant, setTriAscendant] =
    useState(true);
```

```
useEffect(() => {
  if (!isLoggedIn()) {
    router.push("/login");
    return;
  }

  const charger = async () => {
    try {
      const data =
        await getStudents(getToken());
      setUsers(data);
    } catch (error) {
      setErreur(error.message);
    }
    setChargement(false);
  };
  charger();
}, [router]);

if (chargement)
  return <p>Chargement en cours...</p>;
if (erreur)
  return <p className={styles.erreur}>
    Erreur : {erreur}</p>;

const usersFiltres = users
  .filter((u) => {
    const terme = recherche.toLowerCase();
    const nom =
      `${u.prenom} ${u.nom}`.toLowerCase();
    return nom.includes(terme)
      || u.filiere.toLowerCase().includes(terme);
  })
  .sort((a, b) => {
    const na = `${a.nom} ${a.prenom}`;
    const nb = `${b.nom} ${b.prenom}`;
    return triAscendant
      ? na.localeCompare(nb)
      : nb.localeCompare(na);
  });

return (
  <div>
    <h1 className={styles.titre}>
      Liste des étudiants
    </h1>
    <SearchBar recherche={recherche}
               onRecherche={setRecherche} />
    <div className={styles.barre}>
      <span className={styles.compteur}>
        {usersFiltres.length} étudiant(s)
      </span>
      <button className={styles.boutonTri}
        onClick={() =>
          setTriAscendant(!triAscendant)
        }>
        {triAscendant ? "A → Z" : "Z → A"}
```

```
        </button>
      </div>
      {usersFiltres.map((student) => (
        <UserCard key={student.id}
                  student={student} />
      ))}
    </div>
  );
}
```

Ce qui a changé par rapport au Lab 5

1. Le fetch direct est remplacé par `getStudents(getToken())` (module centralisé + token).
2. `isLoggedIn()` vérifie que l'utilisateur est connecté. Sinon, redirection vers `/login`.
3. Le filtre cherche dans `prenom + nom` et dans `filiere` (au lieu de `name`).
4. Le prop est `student` (au lieu de `user`).

5.3 Modifier la page de détail

Remplacez le contenu de `src/app/users/[id]/page.js` :

```
"use client";

import { useState, useEffect } from "react";
import { useParams, useRouter } from "next/navigation";
import Link from "next/link";
import { getStudent, deleteStudent } from "@/lib/api";
import { getToken, isLoggedIn } from "@/lib/auth";
import styles from "./page.module.css";

export default function UserDetail() {
  const { id } = useParams();
  const router = useRouter();
  const [student, setStudent] = useState(null);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);

  useEffect(() => {
    if (!isLoggedIn()) {
      router.push("/login");
      return;
    }

    const charger = async () => {
      try {
        const data =
          await getStudent(getToken(), id);
        setStudent(data);
      } catch (error) {
        setErreur(error.message);
      }
      setChargement(false);
    };
    charger();
  }, [id, router]);
```

```
  const handleDelete = async () => {
    if (!confirm("Supprimer cet étudiant ?"))
      return;
    try {
      await deleteStudent(getToken(), id);
      router.push("/users");
    } catch (error) {
      setErreur(error.message);
    }
  };

  if (chargement) return <p>Chargement...</p>;
  if (erreur) return (
    <div>
      <p>{erreur}</p>
      <Link href="/users">Retour</Link>
    </div>
  );

  return (
    <div>
      <Link href="/users" className={styles.retour}>←
         Retour à la liste
      </Link>
      <div className={styles.profil}>
        <div className={styles.avatar}>
          {student.prenom.charAt(0)}
          {student.nom.charAt(0)}
        </div>
        <h1 className={styles.nom}>
          {student.prenom} {student.nom}
        </h1>
        <span className={styles.badge}>
          {student.filiere} - {student.niveau}
        </span>
      </div>
      <div className={styles.carte}>
        <h2 className={styles.section}>
          Informations
        </h2>
        <div className={styles.champ}>
          <span className={styles.label}>Email</span>
          <span>{student.email}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>Tél.</span>
          <span>{student.telephone || "-"}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>
            Naissance
          </span>
          <span>{student.date_naissance}</span>
        </div>
      </div>
      <button className={styles.boutonSupprimer}
              onClick={handleDelete}>
```

```
        Supprimer cet étudiant
      </button>
    </div>
  );
}
```

La page de détail est devenue un composant client

Au Lab 5, cette page était un **composant serveur** (pas de `"use client"`, fetch avec `await` directement). Maintenant, elle est un **composant client** car elle a besoin de `getToken()` (qui lit `sessionStorage`, côté client uniquement) et de `handleDelete` (événement `onClick`).
C'est un compromis : on perd le rendu serveur pour cette page, mais on gagne l'authentification et la suppression.

Ajoutez les classes `.badge` et `.boutonSupprimer` dans le CSS de la page de détail si elles n'y sont pas :

```
.badge {
  display: inline-block;
  background: #dfe6e9;
  padding: 0.25rem 0.75rem;
  border-radius: 20px;
  font-size: 0.85rem;
  font-weight: 500;
}
.boutonSupprimer {
  margin-top: 1rem;
  padding: 0.6rem 1.2rem;
  background: #fff;
  color: #d63031;
  border: 1px solid #d63031;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 500;
}
.boutonSupprimer:hover { background: #ffeaea; }
```

5.4 Modifier le formulaire d'ajout

Remplacez `src/app/users/add/page.js` pour utiliser les champs du modèle Student (prénom, nom, filière, niveau, date de naissance) au lieu des champs JSONPlaceholder. Le formulaire appelle `createStudent(getToken(), form)` et vérifie `isLoggedIn()` au montage.

Les champs du formulaire :

Champ	Type	Obligatoire
`prenom`	Texte	Oui
`nom`	Texte	Oui
`email`	Texte (avec @)	Oui
`date_naissance`	Date	Oui
`filiere`	Select (IABD, SIR)	Oui
`niveau`	Select (L1 à M2)	Oui
`telephone`	Texte	Non

Le principe est identique au Lab 4 : état objet avec `useState`, `handleChange` avec clé dynamique, validation, `e.preventDefault()`, et redirection avec `router.push()` après succès.

5.5 Modifier la Navbar

Ajoutez un bouton de déconnexion dans `src/components/Navbar.jsx`. Importez `isLoggedIn` et `removeToken` depuis `@/lib/auth`. Si l'utilisateur est connecté, affichez « Déconnexion » (qui appelle `removeToken()` puis redirige vers `/`). Sinon, affichez un lien vers `/login`.

6 Partie 6 — Tester la persistance

Objectif — 5 minutes

Vérifier que le CRUD fonctionne avec de vraies données persistées.

1. Connectez-vous avec `prof` et le mot de passe que `setup_django.sh` a affiché.
2. La liste affiche les 10 étudiants sénégalais.
3. Allez sur `/users/add`. Ajoutez un étudiant (Ibrahima Sarr, SIR, L2).
4. Retournez sur la liste. **Le nouvel étudiant apparaît.**
5. Rechargez la page (`F5`). **Il est toujours là.** Les données sont en base.
6. Cliquez sur un étudiant, puis sur « Supprimer ». Confirmez.
7. La liste se recharge. **L'étudiant a disparu.**

Avant / après : Lab 5 vs Lab 6

	Lab 5	**Lab 6**
Source	JSONPlaceholder (fictif)	Django REST (réel)
Persistance	Aucune	Base de données SQLite
Auth	Aucune	Token (login/logout)
CRUD	Read + faux POST	Read, Create, Delete
Config URL	En dur	Variable d'environnement
Appels API	Dispersés	Centralisés (lib/api.js)

7 Aide-mémoire

Concept	Syntaxe
Variable d'env.	`NEXT_PUBLIC_API_URL=...` dans `.env.local`
Y accéder	`process.env.NEXT_PUBLIC_API_URL`
GET avec token	`headers: { Authorization: `Token ${t}` }`
POST	`method: "POST"` + headers + body
DELETE	`method: "DELETE"` + headers
Stocker le token	`sessionStorage.setItem("token", t)`
Lire le token	`sessionStorage.getItem("token")`
Supprimer	`sessionStorage.removeItem("token")`
Protéger une page	`if (!isLoggedIn()) router.push("/login")`
Module centralisé	`src/lib/api.js`

8 Résumé et conclusion

Dans ce lab, vous avez :

1. Installé et exploré une **API Django REST** pré-remplie.
2. Compris **CORS** et pourquoi le navigateur contrôle les requêtes cross-origin.
3. Créé des **variables d'environnement** et un **module centralisé** (`lib/api.js`).
4. Implémenté l'**authentification par token** : login, stockage, envoi, déconnexion.
5. Adapté le frontend au modèle **Student** (prenom, nom, filiere, niveau...).
6. Vérifié le **CRUD complet** avec des données **persistées**.

Étape	Statut	Ce que vous savez faire
DOM et événements	✓	Sélecteurs, événements
JS moderne	✓	Fonctions fléchées, async/await
React — les bases	✓	Composant, JSX, props, état
useEffect	✓	Montage, dépendances
Router et formulaires	✓	Routes, Link, inputs contrôlés
Next.js App Router	✓	Routing fichier, layout, server/client
API Django	✓	CORS, auth, CRUD, lib/api.js

Bilan du cours

En 7 séances, vous êtes passés de la manipulation manuelle du DOM à une application Next.js complète connectée à un backend Django. Vous maîtrisez le JavaScript moderne, React (composants, état, effets, formulaires), Next.js (App Router, server/client) et la consommation d'API REST (CORS, authentification, CRUD).

Lab 6 — Consommer une API Django — Terminé ✓

Vous savez connecter un frontend Next.js à un backend Django,
gérer l'authentification et réaliser un CRUD complet.

Du DOM manuel à la production : 7 labs, un projet.